



UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática



UNIDADE – FACULDADE DE COMPUTAÇÃO E INFORMÁTICA		
CURSO – CIENCIA DA COMPUTAÇÃO		
COMPONENTE CURRICULAR COMPILADORES		CÓDIGO ENEX01047
PROFESSOR(ES) DANIELA VIEIRA CUNHA FABIO APARECIDO GAMARRA LUBACHESKI LEANDRO CARLOS FERNANDES	DRT 1132090 1146330 1165694	ETAPA 6º
CARGA HORÁRIA 4 h/a (2 teoria 0 laboratório 2 EAD)		SEMESTRE LETIVO 2026 / 2
EMENTA Estudo das fases dos processos de compilação e linguagens. Estudo dos esquemas de análise léxica, análise sintática, análise semântica, geração de código intermediário, alocação de registradores e geração de código Assembly. Estudo comparativo de ferramentas de geração de compiladores.		
OBJETIVOS		
Fatos e Conceitos	Procedimentos e Habilidades	Atitudes, Normas e Valores
• Apresentar os conceitos e algoritmos fundamentais para construção de compiladores de linguagens. • Utilizar ferramentas automáticas para geração de analisadores. • Construir um compilador completo para uma pequena linguagem livre de contexto.	• Descrever a organização arquitetural dos compiladores e seu funcionamento. • Descrever as técnicas de correção de erros utilizadas nos compiladores. • Identificar as formas de geração e de representação de código intermediário. • Identificar, avaliar e utilizar ferramentas de apoio na construção de compiladores.	• Compreender os aspectos ligados ao projeto de linguagens de programação. • Compreender e implementar os principais algoritmos de análise léxica. • Compreender e implementar os principais algoritmos de análise sintática. • Compreender e implementar os processos de análise semântica. • Compreender as técnicas de otimização de código.
CONTEÚDO PROGRAMÁTICO 1. Fundamentos do processo de compilação 1.1. Noções elementares de linguagens formais 1.2. Compilação x interpretação 1.3. Módulos e interfaces de um compilador 1.4. Ferramentas de geração automática de analisadores 2. Análise léxica 2.1. Tokens léxicos 2.2. Expressões regulares 2.3. Autômatos finitos 2.4. Geradores de analisadores léxicos 3. Análise sintática 3.1. Gramáticas livres de contexto 3.2. Análise preditiva 3.3. Análises LL e LR 3.4. Geradores de analisadores sintáticos		



4. Sintaxe abstrata
 - 4.1. Ações semânticas
 - 4.2. Árvores de sintaxe abstrata (AST)
 - 4.3. Percursos em AST e o padrão Visitor
5. Análise semântica
 - 5.1. Tabelas de símbolos
 - 5.2. Verificação de tipos
6. Registros de Ativação
 - 6.1. Organização de programas em tempo de execução
 - 6.2. Stack frames
7. Geração de código intermediário
 - 7.1. Máquinas virtuais
 - 7.2. Árvores de representação intermediárias
 - 7.3. Geração de árvores de representação intermediárias
 - 7.4. Árvores canônicas
8. Geração de código Assembly
 - 8.1. Seleção de instruções
 - 8.2. Análise de sobrevivência
 - 8.3. Alocação de registradores
 - 8.4. Padrões de geração de código

METODOLOGIA

- Aulas expositivas
- Utilização do Ambiente Virtual Mackenzie (Moodle)
- Listas de Exercícios e Projetos Práticos
- Desenvolvimento de um compilador para uma linguagem comercial ou modificada, a ser proposta durante a disciplina.
- Sobre o uso de ferramentas de Inteligência Artificial: nesta disciplina, as ferramentas de Inteligência Artificial (*ChatGPT*, *Gemini Notebook*, *ClaudeCode*, *Antigravity*, etc) são vistas como elemento de apoio à aprendizagem e nunca como substituto do pensamento analítico e crítico. O uso nas atividades propostas é RESTRITO, devendo o aluno informar obrigatoriamente quais ferramentas foram usadas e como auxiliaram no processo (ex: brainstorming, definição de arquitetura, documentação, casos de testes). O uso de IA para gerar integralmente qualquer um dos artefatos, módulos, códigos ou textos dos trabalhos será considerado violação de integridade e desvio da conduta acadêmica esperada ao discente. O conteúdo final, a veracidade das informações e as conclusões são de inteira responsabilidade do aluno.

CRITÉRIO DE AVALIAÇÃO

Nota intermediária 1 (**N1**) composta de:

- Atividade avaliativa I: escrita, individual e sem consulta (P1) (80%)
- Projeto 1: prático, implementação (Prj1) (20%)



Nota intermediária 2 (**N2**) composta de:

- Atividade avaliativa 2: escrita, individual e sem consulta (P2) (70%)
- Projeto 2: prático, implementação (Prj2) (30%)

A Média Intermediária (MI) é dada por

$$MI = \frac{N1 + N2}{2} + NP$$

- *Nota de participação* (NP): pode totalizar até 0.5 ponto (valor de 0 a 0.5), sendo derivada e com valor proporcional a nota da Prova Integrada. A existência desse fator na média está condicionada a aplicação, por parte da Instituição, da Prova Integrada (AvaliA) no semestre corrente.

Critério de aprovação:

Conforme estabelecido no regulamento acadêmico vigente.

BIBLIOGRAFIA BÁSICA

CAMPBELL, B., IYER, S., DELIBAS, B.A. **Introduction to Compiler Construction in a Java World**. New York: Chapman and Hall/CRC, 2012.

GRUNE, D., van REEUWIJK, K., BAL, H.E., JACOBS, C.J.H., LANGENDOEN. **Modern Compiler Design**. New York: Springer, 2012.

REIS, A.J. **Compiler Construction Using Java, JavaCC and YaCC**. New York: Wiley-IEEE, 2011.

BIBLIOGRAFIA COMPLEMENTAR

AHO, A.V., LAM, M.S., SETHI, R., ULLMAN, J.D. **Compilers: Principles, Techniques and Tools**. 2.ed. New York: Addison-Wesley, 2006.

APPEL, A.W. **Modern Compiler Implementation in Java**. 2.ed. Boston: Cambridge University Press, 2002.

COOPER, K., TORCZON, L. **Engineering a Compiler**. 2.ed. New York: Morgan Kaufmann, 2011.

MAK, R. **Writing Compilers and Interpreters: An Engineering Approach**. New York: Wiley, 2009.

WILHELM, R. **Compiler Design: Virtual Machines**. New York: Springer, 2011.